

SUBSYSTEM 3 · PHASE 1 · COMPLETED

Automated Mitigation Layer

LogiTwin AI Engine — Subsystem 3 Complete Documentation
Hook registry · HTTPX async client · Reroute finder · Neo4j audit trail

6	4	27	LIVE
New Files	New Endpoints	Tests Passing	Mitigations Fired

HTTPX 0.28	asyncio.gather	Hook Registry	Circuit Breaker	Pydantic v2	Neo4j Audit	Exception Isolation
------------	----------------	---------------	-----------------	-------------	-------------	---------------------

01 — What Subsystem 3 Added to LogiTwin

SS1 stored data. SS2 detected and scored disruptions. SS3 acts on them automatically — finding reroute options, firing carrier API hooks, writing audit trails to Neo4j, and updating every disruption event status from 'pending_mitigation' to 'mitigated'.

The Mitigation Pipeline

Step	What Happens	Tool
1. Query options	GET /mitigation/options/{event_id} fetches event + builds ranked candidate list	Carriers + Neo4j
2. Rank candidates	Hook registry + alternate path finder combined into ranked list by hook registry	hook_registry.py
3. Select & execute	POST /mitigation/execute/{event_id} fires the chosen carrier API	hooks/mitigation.py
4. HTTPX call	LogiTwinHookClient fires async HTTP POST with full LogiTwin payload + headers	LogiTwinHookClient
5. Retry on failure	Up to 3 retries with exponential backoff — 2s, 4s, 8s between attempts	LogiTwinHookClient
6. Isolation	One failed hook NEVER crashes others — asyncio.gather(return_exceptions=True)	asyncio.gather
7. Neo4j write	:MITIGATION_ACTION relationship written + event status updated to 'mitigated'	neo4j_graph.py
8. Audit trail	Full hook response, duration_ms, carrier_name, action_type stored permanently in graph	neo4j_graph.py

02 — 6 New Files Built

File 1 `app/workers/hook_registry.py`

Config-driven hook map + HTTPX async client

- HookDefinition dataclass — hook_id, node_id, carrier_name, endpoint_url, action_type, priority, eta_delta_days, cost_delta_usd, metadata
- HOOK_REGISTRY dict — maps every supply chain node to its backup carrier hooks. 2 hooks each for port-001, sup-001, fac-001 (6 total)
- LogiTwinHookClient — async HTTPX client with 10s timeout, 3 retries, exponential backoff, dry_run mode, full exception isolation per connector
- fire_hook() — fires a single hook and returns structured result dict. NEVER raises. Always returns success=True or success=False with error detail
- fire_hooks_parallel() — fires multiple hooks via asyncio.gather(return_exceptions=True). One failure never blocks or cancels others

6 hooks registered across 3 nodes. All verified: priorities sorted ascending, ETAs non-negative, endpoints present.

File 2 `app/services/mitigation.py`

Alternate path finder + reroute candidate ranker

- get_disruption_event(event_id) — fetches :DISRUPTION_EVENT node + all :IMPACTS relationships from Neo4j
- find_alternate_paths(disruption_type, node_id) — 3 Cypher queries depending on type: alternate ports, alternate suppliers, alternate factories
- build_mitigation_options(event_id) — combines disruption event + graph paths + hook registry into ranked option list with recommendations
- _build_recommendation() — generates urgency-aware text. 'IMMEDIATELY' for severity >= 0.8, 'SOON' for lower. Carrier-specific language per action type
- write_mitigation_action(event_id, hook_result, selected_option) — writes :MITIGATION_ACTION + updates event status in Neo4j

File 3 `app/routes/mitigation.py`

4 new API endpoints — full mitigation control

Method	Path	Purpose	Returns
GET	/mitigation/options/{event_id}	Get ranked reroute candidates	Ranked options list
POST	/mitigation/execute/{event_id}	Fire selected hook	Full execution result
GET	/mitigation/events	All disruption events + status	Ordered by timestamp
GET	/mitigation/hooks	Full hook registry	All nodes + hooks

File 4 **app/models.py additions**

4 new Pydantic v2 models for mitigation data

Model	Used by	Key fields
MitigationOptionModel	GET /options	option_id, hook_id, priority, eta_delta_days, cost_delta_usd, recommendation
MitigationExecuteRequestModel	POST /execute	hook_id, dry_run (default False), option_data dict
MitigationResultModel	POST /execute response	action_id, success, duration_ms, new_event_status, hook_response
HookSummaryModel	GET /hooks	hook_id, carrier_name, action_type, priority, eta_delta_days, cost_delta_usd

File 5 **tests/test_mitigation.py**

27 unit tests — all passing in 3.40 seconds

Test Class	Tests	What It Verifies	Result
TestHookRegistry	12	All nodes have hooks, priorities sorted, lookup by ID, empty node	12/12 PASS
TestHookClientDryRun	5	Dry run success, payload echo, parallel fire, exception isolation	5/5 PASS
TestRecommendationBuilder	6	Urgency text, carrier names, team names, severity thresholds	6/6 PASS
TestMitigationModels	4	Pydantic v2 instantiation, dry_run default, metadata default	4/4 PASS

03 — Live Mitigation Results

Two mitigations executed live against the real LogiTwin graph. One dry run, one real HTTP call to httpbin.org simulating a carrier API.

Event	Hook Fired	Mode	Success	Duration	New Status
evt-cea9 (port_congestion)	hook-port001-maersk-expedite	DRY RUN	true	0ms	mitigated
evt-64e7 (supplier_offline)	hook-sup001-backup-activate	LIVE	true	10,026ms	mitigated
evt-9ff3 (capacity_degradation)	pending	—	—	—	pending

LIVE call proof: httpbin.org returned HTTP 200 with full payload echo. X-LogiTwin-Hook and X-Action-Type headers confirmed received at carrier endpoint. Real carrier API integration pattern validated end-to-end.

04 — Registered Hook Registry

Node	Hook ID	Carrier	Action	ETA+	Cost
port-001	hook-port001-maersk-expedite	Maersk Line	expedite_clearance	+1.5d	\$2,500
port-001	hook-port001-dhl-reroute	DHL Supply Chain	reroute_shipment	+4.0d	\$8,500
sup-001	hook-sup001-backup-activate	Bharat Components	activate_backup	+3.0d	\$12,000
sup-001	hook-sup001-procurement-notify	LogiTwin Alert	notify_procurement	+0.0d	\$0
fac-001	hook-fac001-capacity-notify	LogiTwin Ops Alert	notify_procurement	+0.0d	\$0
fac-001	hook-fac001-fedex-expedite	FedEx Freight	reroute_shipment	+2.0d	\$5,500

05 — Final Neo4j Graph Schema — All 3 Subsystems

Element	Properties	Added in
(:Supplier)	id, name, tier, location	SS1
(:Factory)	id, name, capacity_rate, location	SS1
(:Port)	id, name, average_clearance_days, status	SS1
(:DISRUPTION_EVENT)	id, disruption_type, disrupted_node_id, severity_input, highest_risk_level, timestamp, status, mitigated_at	SS2
[SUPPLIES]	component_id, lead_time_days	SS1

Element	Properties	Added in
[SHIPS_THROUGH]	transit_days	SS1
[DELIVERS_TO]	transit_days	SS1
[IMPACTS]	impact_score, risk_level, path_type	SS2
[MITIGATION_ACTION]	action_id, hook_id, carrier_name, action_type, success, eta_delta_days, cost_delta_usd, executed_at, duration_ms, error	SS3 NEW

06 — Startup Commands — All 3 Subsystems Active

Terminal 1 — Databases

```
docker start logitwin-neo4j logitwin-redis
docker ps | grep -E 'neo4j|redis'
```

Terminal 2 — FastAPI

```
cd ~/projects/logitwin && source .venv/bin/activate
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Terminal 3 — Celery

```
cd ~/projects/logitwin && source .venv/bin/activate
celery -A app.workers.celery_app.celery_app worker --loglevel=info --concurrency=4
```

Verify all live

```
curl http://localhost:8000/health
curl -s http://localhost:8000/mitigation/hooks | python3 -m json.tool
```

07 — Subsystem 4 Preview: Productization & Demo Layer

SS4 transforms LogiTwin from a working engine into a demo-ready product that can be shown to investors and clients.

- JWT-based API authentication — role-based access: admin vs read-only viewer roles
- OpenAPI spec polish — clean interactive docs at /docs for enterprise client integrations
- Real-time disruption alert stream via Server-Sent Events (SSE)
- Streamlit dashboard — live graph visualization of supply network and active disruptions
- Demo seed script — Taiwan port strike: 3 factories at risk, auto-reroute fires end-to-end
- GitHub Actions CI — lint, test, Docker build on every push to main
- README + architecture diagram + one-command docker compose up startup for demos
- End-to-end integration tests covering the full ingest → simulate → mitigate pipeline

Resume command for next session:

"Resume LogiTwin — Subsystem 3 complete, begin Subsystem 4: Productization and Demo Layer"